Zcash Wallet Adapter Acceptance Specification

Subject: WalletConnect support in @rhea-finance/zcash-wallet-adapter. Baseline version: 0.2.3. Date: 2026-11-05.

1. Scope and Usage

This specification covers package installation, interfaces, state management, and WalletConnect interoperability. It does not depend on a particular website, deployment URL, or reference wallet page. Run acceptance tests in an application integrating this adapter version, with a wallet that implements this Zcash WalletConnect protocol.

WalletConnect is the connection protocol. The adapter's current WalletConnect transport supports Zcash Mainnet; it does not add Ethereum, other chains, or PCZT signing. The wallet handles authorization, fees, coin selection, shielded proofs, signing, and broadcasting. The adapter maps requests and manages application-side state.

Install the published package:

```
pnpm add @rhea-finance/zcash-wallet-adapter@0.2.3
pnpm list @rhea-finance/zcash-wallet-adapter --depth 0
```

Expected: installation resolves to 0.2.3. The application build resolves the root entry, /walletconnect, /react, and styles.css. WalletConnectZcashWalletAdapter is exported from the /walletconnect subpath.

Usage sequence: configure projectId and application metadata; register WalletConnectZcashWalletAdapter; let the user initiate connection; display the pairing URI or QR; open or scan it in the wallet; obtain approval; read accounts and balances; request signing or a transfer; obtain wallet confirmation; receive the adapter result.

For React, register a stable adapters array with WalletProvider. Inside its children, useWallet() exposes connection state, addresses, connect, disconnect, signMessage, and signAndSendTransaction. Pass the adapter instance's name when connecting. For the built-in selector, import WalletProvider from the root entry and load styles.css. For custom UI, use the headless Provider from /react.

Query balances through the current WalletConnect adapter's getBalance(), and a single address through getAddress(). These methods are not directly exposed by useWallet(). Non-React applications can call the adapter directly or manage it through WalletStore.

Prerequisites: a valid Reown projectId, network access to WalletConnect services, and an unlocked wallet supporting the four Zcash methods. Transfer tests additionally require a small amount of mainnet ZEC, transaction fees, and a valid recipient address controlled by the tester. Keep test funds separate from everyday funds. Mock tests cannot replace real transaction tests.

2. Connection, Accounts, and Balances

WC-01 Establish a WalletConnect Session

Action: select WalletConnect in the application and call connect(), or connect(adapter.name) through the Hook. Handle displayUri or onDisplayUri to show pairing information. Open the URI or scan the QR in a compatible wallet and approve.

Pass: a valid pairing URI is produced; after approval, the connection Promise resolves, isConnected is true, connect is emitted, and exposed addresses belong to the authorized account. The application chain is zcash:mainnet. A QR code or an opened wallet alone is insufficient. Record adapter.name to confirm the test uses WalletConnect rather than an injected extension connection.

Protocol check: new sessions require namespace bip122 and chain ID bip122:00040fe8ec8471911baa1db1266ea15d. Required methods: zcash_getAddress, zcash_getBalance, zcash_transfer, zcash_signMessage. Required events: accountsChanged, chainChanged. A wallet must not claim approval without satisfying required permissions.

WC-02 Reject a Connection

Action: disconnect an existing session, initiate a new connection, and reject authorization in the wallet.

Pass: connection fails, isConnected is false, no authorized account state is established, and connection can be retried. The application can catch the failure; exact UI wording is not prescribed.

WC-03 Accounts and Addresses

Action: authorize one account containing shielded and transparent addresses. Read shieldedAddress and transparentAddress, then call getAddress().

Pass: addresses match wallet authorization data; getAddress() returns a valid address. If only one address type is authorized, the other may be empty and must not be fabricated. Session accounts use CAIP-10 format. The session public key is not a Zcash identity public key and must not be used to verify transparent address ownership.

WC-04 Balances

Action: call getBalance() and compare against the same wallet account at the same synchronization state.

Pass: normalized fields include transparent, shielded, total, spendable, and available. Amounts are ZEC decimal strings without floating-point precision loss. Compare equivalent balance categories; available need not equal total. Automated tests can check missing-field fallbacks, but those fallbacks do not prove the accuracy of balances the wallet did not supply.

3. Message Signing and Transactions

WC-05 Approve Message Signing

Action: call signMessage through the Hook or adapter with the message "WalletConnect acceptance 2026-11-05 test" and signingMode set to derived. Review the original text in the wallet and approve. Also test non-ASCII UTF-8 text.

Pass: the request uses zcash_signMessage and preserves the message and mode. The result includes signature, publicKey, address, and signingMode. The default mode is derived. Use the consuming application's verifier to check the signature, public key, address, and original message; a modified message must fail verification. The adapter does not verify signatures cryptographically. A non-empty string proves only that a response was returned.

WC-06 Reject Message Signing

Action: initiate another signing request and reject it in the wallet.

Pass: the Promise rejects with an application-readable error, no successful result is returned for this request, and the connection remains usable. Signing through an older session without zcash_signMessage must explicitly fail as unsupported.

WC-07 Approve a Transfer

Action: call signAndSendTransaction with a real mainnet recipient, amount "0.001", and fundingSource shielded. Review the address, amount, funding source, and fees in the wallet before approval. Repeat separately with transparent funding.

Pass: the request uses zcash_transfer. The fundingSource and compatibility field type match; the default funding source is shielded. After wallet approval and broadcast, the adapter returns a txid string. Record broadcast and confirmation separately, and verify receipt in the recipient wallet after confirmation. Shielded details need not be publicly visible in an explorer. Record each funding source independently.

WC-08 Reject a Transfer and Test Insufficient Funds

Action: reject a valid transfer in the wallet. Separately, request an amount exceeding the selected funding pool's available balance.

Pass: both return catchable errors, without a successful txid, a corresponding outgoing transfer, or a silent funding-source change. Rejection and insufficient funds do not clear a valid connection; later operations can be retried. Wallet errors, including code 5000, remain identifiable by the application.

WC-09 Invalid Inputs and Unsupported Capabilities

Action: separately submit an empty address, zero, a negative amount, an amount exceeding eight decimal places, and a memo exceeding 512 UTF-8 bytes. Call signTransaction() for PCZT signing.

Pass: invalid transfer inputs fail adapter validation without sending a transfer request. PCZT signing throws WalletMethodNotSupportedError. The wallet must still validate receiver support for valid memos. Amounts have at most eight decimal places and must not be approximated before submission.

4. Sessions, Errors, and Acceptance Results

WC-10 Restore a Session Silently

Action: keep a valid session, reload the application, and restore through WalletStore.autoConnect(), Provider autoConnect, or connect({ silent: true }).

Pass: the same account is restored without a new pairing URI or authorization request. With no compatible session, silent connect fails without displaying a new QR. Disconnected or expired sessions must not appear restored as usable connections.

WC-11 Disconnect from Either Side

Action: call disconnect() in the application and reconnect. Then end the session from the wallet.

Pass: both paths disconnect the adapter, clear active account state, and emit disconnect. The old connection cannot continue operating. A session_delete for another topic must not disconnect the current session. New authorization establishes a working connection.

WC-12 Account and Network Events; Error Isolation

Action: use an event-capable wallet or protocol test tool to send accountsChanged and chainChanged for the current session. Send an event for another topic, and simulate a wallet RPC failure.

Pass: supported account events update the account and emit a change; a non-mainnet change reports unsupported-chain behavior. Other topics do not affect the connection. Signing and transfer failures produce handleable errors without automatically clearing a valid connection. The wallet must independently reject network-mismatched transactions.

Known limits: accountsChanged reads only the first array item and cannot refresh a complete shielded/transparent address pair together. Disconnect and reauthorize to test a complete account switch. Existing-session compatibility requires only address, balance, and transfer methods, while new proposals require all four. Restoration compatibility is not full support for a new integration.

Acceptance Records

For each WC case, record Pass, Fail, or Blocked, actual behavior, and evidence. Record the exact adapter version, application build, wallet version, browser/OS, and time. For transfers, record source, amount, txid, broadcast, confirmation, and receipt. For signatures, record the original message, mode, and verification result. Never record private keys, recovery phrases, or complete pairing URIs.

Acceptance Criteria

Execute every case and meet its pass criteria. Protocol simulations verify input, event, and error boundaries; real-wallet tests verify authorization, signatures, and actual transactions. Unfinished tests, connectivity issues, or insufficient test funds are Blocked, not Pass. Results do not cover untested wallets, chains, or capabilities.

For technical review in the adapter source repository, run pnpm install, pnpm typecheck, pnpm test, and pnpm build. Passing automated checks does not establish real on-chain acceptance. This document is a test specification, not an executed test report.